

10. Basic concepts

10.1 Application startup

Application startup occurs when the execution environment calls a designated method, which is referred to as the application's **entry point**. This entry point method is always named `Main`, and shall have one of the following signatures:

```
static void Main() {...}
static void Main(string[] args) {...}
static int Main() {...}
static int Main(string[] args) {...}
```

As shown, the entry point may optionally return an `int` value. This return value is used in application termination (§10.2).

The entry point may optionally have one formal parameter, and this formal parameter may have any name. If such a parameter is declared, it must obey the following constraints:

- The implementation shall ensure that the value of this parameter is not `null`.
- Let `args` be the name of the parameter. If the length of the array designated by `args` is greater than zero, the array members `args[0]` through `args[args.Length-1]`, inclusive, must refer to strings, called **application parameters**, which are given implementation-defined values by the host environment prior to application startup. The intent is to supply to the application information determined prior to application startup from elsewhere in the hosted environment. If the host environment is not capable of supplying strings with letters in both uppercase and lowercase, the implementation shall ensure that the strings are received in lowercase. [Note: On systems supporting a command line, application parameters correspond to what are generally known as command-line arguments. *end note*]

Since C# supports method overloading, a class or struct may contain multiple definitions of some method, provided each has a different signature. However, within a single program, no class or struct shall contain more than one method called `Main` whose definition qualifies it to be used as an application entry point. Other overloaded versions of `Main` are permitted, however, provided they have more than one parameter, or their only parameter is other than type `string[]`.

An application can be made up of multiple classes or structs. It is possible for more than one of these classes or structs to contain a method called `Main` whose definition qualifies it to be used as an application entry point. In such cases, one of these `Main` methods must be chosen as the entry point so that application startup can occur. This choice of an entry point is beyond the scope of this specification—no mechanism for specifying or determining an entry point is provided.

In C#, every method must be defined as a member of a class or struct. Ordinarily, the declared accessibility (§10.5.1) of a method is determined by the access modifiers (§17.2.3) specified in its declaration, and similarly the declared accessibility of a type is determined by the access modifiers specified in its declaration. In order for a given method of a given type to be callable, both the type and the member must be accessible. However, the application entry point is a special case. Specifically, the execution environment can access the application's entry point regardless of its declared accessibility and regardless of the declared accessibility of its enclosing type declarations.

In all other respects, entry point methods behave like those that are not entry points.

10.2 Application termination

Application termination returns control to the execution environment.

If the return type of the application's entry point method is `int`, the value returned serves as the application's **termination status code**. The purpose of this code is to allow communication of success or failure to the execution environment.

If the return type of the entry point method is `void`, reaching the right brace (`}`) which terminates that method, or executing a `return` statement that has no expression, results in a termination status code of 0.

Prior to an application's termination, destructors for all of its objects that have not yet been garbage collected are called, unless such cleanup has been suppressed (by a call to the library method `GC.SuppressFinalize`, for example).

10.3 Declarations

Declarations in a C# program define the constituent elements of the program. C# programs are organized using namespaces (§16), which can contain type declarations and nested namespace declarations. Type declarations (§16.5) are used to define classes (§17), structs (§18), interfaces (§20), enums (§21), and delegates (§22). The kinds of members permitted in a type declaration depend on the form of the type declaration. For instance, class declarations can contain declarations for constants (§17.3), fields (§17.4), methods (§17.5), properties (§17.6), events (§17.7), indexers (§17.8), operators (§17.9), instance constructors (§17.10), destructors (§17.12), static constructors (§17.11), and nested types.

A declaration defines a name in the **declaration space** to which the declaration belongs. Except for overloaded members (§10.6), it is a compile-time error to have two or more declarations that introduce members with the same name in a declaration space. However, no diagnostic is required if the declaration space is a namespace for the global declaration space and the conflicting declarations are in separate programs. It is never possible for a declaration space to contain different kinds of members with the same name. For example, a declaration space can never contain a field and a method by the same name.

There are several different types of declaration spaces, as described in the following.

- Within all source files of a program, *namespace-member-declarations* with no enclosing *namespace-declaration* are members of a single combined declaration space called the **global declaration space**.
- Within all source files of a program, *namespace-member-declarations* within *namespace-declarations* that have the same fully qualified namespace name are members of a single combined declaration space.
- Each class, struct, or interface declaration creates a new declaration space. Names are introduced into this declaration space through *class-member-declarations*, *struct-member-declarations*, or *interface-member-declarations*. Except for overloaded instance constructor declarations and static constructor declarations, a class or struct member declaration cannot introduce a member by the same name as the class or struct. A class, struct, or interface permits the declaration of overloaded methods and indexers. Furthermore, a class or struct permits the declaration of overloaded instance constructors and operators. For example, a class, struct, or interface may contain multiple method declarations with the same name, provided these method declarations differ in their signature (§10.6). Note that base classes do not contribute to the declaration space of a class, and base interfaces do not contribute to the declaration space of an interface. Thus, a derived class or interface is allowed to declare a member with the same name as an inherited member. Such a member is said to **hide** the inherited member.
- Each enumeration declaration creates a new declaration space. Names are introduced into this declaration space through *enum-member-declarations*.
- Each *block* or *switch-block* creates a different declaration space for local variables. Names are introduced into this declaration space through *local-variable-declarations*. If a block is the body of an instance constructor, method, or operator declaration, or a get or set accessor for an indexer declaration, the parameters declared in such a declaration are members of the block's **local variable declaration space**. The local variable declaration space of a block includes any nested blocks. Thus, within a nested block it is not possible to declare a local variable with the same name as a local variable in an enclosing block.
- Each *block* or *switch-block* creates a separate declaration space for labels. Names are introduced into this declaration space through *labeled-statements*, and the names are referenced through *goto-statements*.

The **label declaration space** of a block includes any nested blocks. Thus, within a nested block it is not possible to declare a label with the same name as a label in an enclosing block.

The textual order in which names are declared is generally of no significance. In particular, textual order is not significant for the declaration and use of namespaces, constants, methods, properties, events, indexers, operators, instance constructors, destructors, static constructors, and types. Declaration order is significant in the following ways:

- Declaration order for field declarations and local variable declarations determines the order in which their initializers (if any) are executed.
- Local variables must be defined before they are used (§10.7).
- Declaration order for enum member declarations (§21.3) is significant when *constant-expression* values are omitted.

[*Example:* The declaration space of a namespace is “open ended”, and two namespace declarations with the same fully qualified name contribute to the same declaration space. For example

```

namespace Megacorp.Data
{
    class Customer
    {
        ...
    }
}

namespace Megacorp.Data
{
    class Order
    {
        ...
    }
}

```

The two namespace declarations above contribute to the same declaration space, in this case declaring two classes with the fully qualified names `Megacorp.Data.Customer` and `Megacorp.Data.Order`. Because the two declarations contribute to the same declaration space, it would have caused a compile-time error if each contained a declaration of a class with the same name. *end example*]

[*Note:* As specified above, the declaration space of a block includes any nested blocks. Thus, in the following example, the `F` and `G` methods result in a compile-time error because the name `i` is declared in the outer block and cannot be redeclared in the inner block. However, the `H` and `I` methods are valid since the two `i`'s are declared in separate non-nested blocks.

```

class A
{
    void F() {
        int i = 0;
        if (true) {
            int i = 1;
        }
    }

    void G() {
        if (true) {
            int i = 0;
        }
        int i = 1;
    }
}

```

```

1      void H() {
2          if (true) {
3              int i = 0;
4          }
5          if (true) {
6              int i = 1;
7          }
8      }
9
10     void I() {
11         for (int i = 0; i < 10; i++)
12             H();
13         for (int i = 0; i < 10; i++)
14             H();
15     }
16 end note]

```

10.4 Members

Namespaces and types have **members**. [Note: The members of an entity are generally available through the use of a qualified name that starts with a reference to the entity, followed by a “.” token, followed by the name of the member. *end note*]

Members of a type are either declared in the type or **inherited** from the base class of the type. When a type inherits from a base class, all members of the base class, except instance constructors, destructors, and static constructors become members of the derived type. The declared accessibility of a base class member does not control whether the member is inherited—inheritance extends to any member that isn’t an instance constructor, static constructor, or destructor. However, an inherited member may not be accessible in a derived type, either because of its declared accessibility (§10.5.1) or because it is hidden by a declaration in the type itself (§10.7.1.2).

10.4.1 Namespace members

Namespaces and types that have no enclosing namespace are members of the **global namespace**. This corresponds directly to the names declared in the global declaration space.

Namespaces and types declared within a namespace are members of that namespace. This corresponds directly to the names declared in the declaration space of the namespace.

Namespaces have no access restrictions. It is not possible to declare private, protected, or internal namespaces, and namespace names are always publicly accessible.

10.4.2 Struct members

The members of a struct are the members declared in the struct and the members inherited from class **object**.

The members of a simple type correspond directly to the members of the struct type aliased by the simple type:

- The members of `sbyte` are the members of the `System.SByte` struct.
- The members of `byte` are the members of the `System.Byte` struct.
- The members of `short` are the members of the `System.Int16` struct.
- The members of `ushort` are the members of the `System.UInt16` struct.
- The members of `int` are the members of the `System.Int32` struct.
- The members of `uint` are the members of the `System.UInt32` struct.
- The members of `long` are the members of the `System.Int64` struct.
- The members of `ulong` are the members of the `System.UInt64` struct.
- The members of `char` are the members of the `System.Char` struct.
- The members of `float` are the members of the `System.Single` struct.
- The members of `double` are the members of the `System.Double` struct.
- The members of `decimal` are the members of the `System.Decimal` struct.
- The members of `bool` are the members of the `System.Boolean` struct.

10.4.3 Enumeration members

The members of an enumeration are the constants declared in the enumeration and the members inherited from class `object`.

10.4.4 Class members

The members of a class are the members declared in the class and the members inherited from the base class (except for class `object` which has no base class). The members inherited from the base class include the constants, fields, methods, properties, events, indexers, operators, and types of the base class, but not the instance constructors, destructors, and static constructors of the base class. Base class members are inherited without regard to their accessibility.

A class declaration may contain declarations of constants, fields, methods, properties, events, indexers, operators, instance constructors, destructors, static constructors, and types.

The members of `object` and `string` correspond directly to the members of the class types they alias:

- The members of `object` are the members of the `System.Object` class.
- The members of `string` are the members of the `System.String` class.

10.4.5 Interface members

The members of an interface are the members declared in the interface and in all base interfaces of the interface, and the members inherited from class `object`.

10.4.6 Array members

The members of an array are the members inherited from class `System.Array`.

10.4.7 Delegate members

The members of a delegate are the members inherited from class `System.Delegate`.

10.5 Member access

Declarations of members allow control over member access. The accessibility of a member is established by the declared accessibility (§10.5.1) of the member combined with the accessibility of the immediately containing type, if any.

When access to a particular member is allowed, the member is said to be **accessible**. Conversely, when access to a particular member is disallowed, the member is said to be **inaccessible**. Access to a member is permitted when the textual location in which the access takes place is included in the accessibility domain (§10.5.2) of the member.

10.5.1 Declared accessibility

The **declared accessibility** of a member can be one of the following:

- Public, which is selected by including a `public` modifier in the member declaration. The intuitive meaning of `public` is “access not limited”.
- Protected, which is selected by including a `protected` modifier in the member declaration. The intuitive meaning of `protected` is “access limited to the containing class or types derived from the containing class”.
- Internal, which is selected by including an `internal` modifier in the member declaration. The intuitive meaning of `internal` is “access limited to this program”.
- Protected internal, which is selected by including both a `protected` and an `internal` modifier in the member declaration. The intuitive meaning of `protected internal` is “access limited to this program or types derived from the containing class”.
- Private, which is selected by including a `private` modifier in the member declaration. The intuitive meaning of `private` is “access limited to the containing type”.

Depending on the context in which a member declaration takes place, only certain types of declared accessibility are permitted. Furthermore, when a member declaration does not include any access modifiers, the context in which the declaration takes place determines the default declared accessibility.

- Namespaces implicitly have `public` declared accessibility. No access modifiers are allowed on namespace declarations.
- Types declared in compilation units or namespaces can have `public` or `internal` declared accessibility and default to `internal` declared accessibility.
- Class members can have any of the five kinds of declared accessibility and default to `private` declared accessibility. (Note that a type declared as a member of a class can have any of the five kinds of declared accessibility, whereas a type declared as a member of a namespace can have only `public` or `internal` declared accessibility.)
- Struct members can have `public`, `internal`, or `private` declared accessibility and default to `private` declared accessibility because structs are implicitly sealed. Struct members introduced in a struct (that is, not inherited by that struct) cannot have `protected` or `protected internal` declared accessibility. (Note that a type declared as a member of a struct can have `public`, `internal`, or `private` declared accessibility, whereas a type declared as a member of a namespace can have only `public` or `internal` declared accessibility.)
- Interface members implicitly have `public` declared accessibility. No access modifiers are allowed on interface member declarations.
- Enumeration members implicitly have `public` declared accessibility. No access modifiers are allowed on enumeration member declarations.

10.5.2 Accessibility domains

The **accessibility domain** of a member consists of the (possibly disjoint) sections of program text in which access to the member is permitted. For purposes of defining the accessibility domain of a member, a member is said to be **top-level** if it is not declared within a type, and a member is said to be **nested** if it is declared within another type. Furthermore, the text of an assembly is defined as all source text contained in all source files of that assembly, and the source text of a type is defined as all source text contained between the

opening and closing “{” and “}” tokens in the *class-body*, *struct-body*, *interface-body*, or *enum-body* of the type (including, possibly, types that are nested within the type).

The accessibility domain of a predefined type (such as `object`, `int`, or `double`) is unlimited.

The accessibility domain of a top-level type `T` that is declared in a program `P` is defined as follows:

- If the declared accessibility of `T` is `public`, the accessibility domain of `T` is the program text of `P` and any program that references `P`.
- If the declared accessibility of `T` is `internal`, the accessibility domain of `T` is the program text of `P`.

[*Note:* From these definitions it follows that the accessibility domain of a top-level type is always at least the program text of the program in which that type is declared. *end note*]

The accessibility domain of a nested member `M` declared in a type `T` within a program `P`, is defined as follows (noting that `M` itself may possibly be a type):

- If the declared accessibility of `M` is `public`, the accessibility domain of `M` is the accessibility domain of `T`.
- If the declared accessibility of `M` is `protected internal`, let `D` be the union of the program text of `P` and the program text of any type derived from `T`, which is declared outside `P`. The accessibility domain of `M` is the intersection of the accessibility domain of `T` with `D`.
- If the declared accessibility of `M` is `protected`, let `D` be the union of the program text of `T` and the program text of any type derived from `T`. The accessibility domain of `M` is the intersection of the accessibility domain of `T` with `D`.
- If the declared accessibility of `M` is `internal`, the accessibility domain of `M` is the intersection of the accessibility domain of `T` with the program text of `P`.
- If the declared accessibility of `M` is `private`, the accessibility domain of `M` is the program text of `T`.

[*Note:* From these definitions it follows that the accessibility domain of a nested member is always at least the program text of the type in which the member is declared. Furthermore, it follows that the accessibility domain of a member is never more inclusive than the accessibility domain of the type in which the member is declared. *end note*]

[*Note:* In intuitive terms, when a type or member `M` is accessed, the following steps are evaluated to ensure that the access is permitted:

- First, if `M` is declared within a type (as opposed to a compilation unit or a namespace), a compile-time error occurs if that type is not accessible.
- Then, if `M` is `public`, the access is permitted.
- Otherwise, if `M` is `protected internal`, the access is permitted if it occurs within the program in which `M` is declared, or if it occurs within a class derived from the class in which `M` is declared and takes place through the derived class type (§10.5.3).
- Otherwise, if `M` is `protected`, the access is permitted if it occurs within the class in which `M` is declared, or if it occurs within a class derived from the class in which `M` is declared and takes place through the derived class type (§10.5.3).
- Otherwise, if `M` is `internal`, the access is permitted if it occurs within the program in which `M` is declared.
- Otherwise, if `M` is `private`, the access is permitted if it occurs within the type in which `M` is declared.
- Otherwise, the type or member is inaccessible, and a compile-time error occurs.

end note]

[*Example:* In the example

```

1      public class A
2      {
3          public static int X;
4          internal static int Y;
5          private static int Z;
6      }
7
8      internal class B
9      {
10         public static int X;
11         internal static int Y;
12         private static int Z;
13
14         public class C
15         {
16             public static int X;
17             internal static int Y;
18             private static int Z;
19         }
20
21         private class D
22         {
23             public static int X;
24             internal static int Y;
25             private static int Z;
26         }
27     }

```

the classes and members have the following accessibility domains:

- The accessibility domain of A and A.X is unlimited.
- The accessibility domain of A.Y, B, B.X, B.Y, B.C, B.C.X, and B.C.Y is the program text of the containing program.
- The accessibility domain of A.Z is the program text of A.
- The accessibility domain of B.Z and B.D is the program text of B, including the program text of B.C and B.D.
- The accessibility domain of B.C.Z is the program text of B.C.
- The accessibility domain of B.D.X, B.D.Y, and B.D.Z is the program text of B.D.

As the example illustrates, the accessibility domain of a member is never larger than that of a containing type. For example, even though all X members have public declared accessibility, all but A.X have accessibility domains that are constrained by a containing type. *end example*]

As described in §10.4, all members of a base class, except for instance constructors, destructors, and static constructors are inherited by derived types. This includes even private members of a base class. However, the accessibility domain of a private member includes only the program text of the type in which the member is declared. [*Example:* In the example

```

41     class A
42     {
43         int x;
44         static void F(B b) {
45             b.x = 1;    // ok
46         }
47     }
48
49     class B: A
50     {
51         static void F(B b) {
52             b.x = 1;    // Error, x not accessible
53         }
54     }

```


the B class inherits the private member `x` from the A class. Because the member is private, it is only accessible within the *class-body* of A. Thus, the access to `b.x` succeeds in the A.F method, but fails in the B.F method. *end example*]

10.5.3 Protected access for instance members

When a **protected** instance member is accessed outside the program text of the class in which it is declared, and when a **protected internal** instance member is accessed outside the program text of the program in which it is declared, the access is required to take place *through* an instance of the derived class type in which the access occurs. Let B be a base class that declares a protected instance member M, and let D be a class that derives from B. Within the *class-body* of D, access to M can take one of the following forms:

- An unqualified *type-name* or *primary-expression* of the form M.
- A *primary-expression* of the form `E.M`, provided the type of E is D or a class derived from D.
- A *primary-expression* of the form `base.M`.

In addition to these forms of access, a derived class can access a protected instance constructor of a base class in a *constructor-initializer* (§17.10.1).

[*Example:* In the example

```
public class A
{
    protected int x;
    static void F(A a, B b) {
        a.x = 1;    // ok
        b.x = 1;    // ok
    }
}

public class B: A
{
    static void F(A a, B b) {
        a.x = 1;    // Error, must access through instance of B
        b.x = 1;    // ok
    }
}
```

within A, it is possible to access `x` through instances of both A and B, since in either case the access takes place *through* an instance of A or a class derived from A. However, within B, it is not possible to access `x` through an instance of A, since A does not derive from B. *end example*]

10.5.4 Accessibility constraints

Several constructs in the C# language require a type to be **at least as accessible as** a member or another type. A type T is said to be at least as accessible as a member or type M if the accessibility domain of T is a superset of the accessibility domain of M. In other words, T is at least as accessible as M if T is accessible in all contexts in which M is accessible.

The following accessibility constraints exist:

- The direct base class of a class type must be at least as accessible as the class type itself.
- The explicit base interfaces of an interface type must be at least as accessible as the interface type itself.
- The return type and parameter types of a delegate type must be at least as accessible as the delegate type itself.
- The type of a constant must be at least as accessible as the constant itself.
- The type of a field must be at least as accessible as the field itself.
- The return type and parameter types of a method must be at least as accessible as the method itself.
- The type of a property must be at least as accessible as the property itself.
- The type of an event must be at least as accessible as the event itself.
- The type and parameter types of an indexer must be at least as accessible as the indexer itself.
- The return type and parameter types of an operator must be at least as accessible as the operator itself.
- The parameter types of an instance constructor must be at least as accessible as the instance constructor itself.

[*Example:* In the example

```
class A {...}
public class B: A {...}
```

the B class results in a compile-time error because A is not at least as accessible as B. *end example*]

[*Example:* Likewise, in the example

```
class A {...}
public class B
{
    A F() {...}
    internal A G() {...}
    public A H() {...}
}
```

the H method in B results in a compile-time error because the return type A is not at least as accessible as the method. *end example*]

10.6 Signatures and overloading

Methods, instance constructors, indexers, and operators are characterized by their **signatures**:

- The signature of a method consists of the name of the method and the type and kind (value, reference, or output) of each of its formal parameters, considered in the order left to right. The signature of a method specifically does not include the return type, nor does it include the **params** modifier that may be specified for the right-most parameter.
- The signature of an instance constructor consists of the type and kind (value, reference, or output) of each of its formal parameters, considered in the order left to right. The signature of an instance constructor specifically does not include the **params** modifier that may be specified for the right-most parameter.
- The signature of an indexer consists of the type of each of its formal parameters, considered in the order left to right. The signature of an indexer specifically does not include the element type.
- The signature of an operator consists of the name of the operator and the type of each of its formal parameters, considered in the order left to right. The signature of an operator specifically does not include the result type.

Signatures are the enabling mechanism for **overloading** of members in classes, structs, and interfaces:

- Overloading of methods permits a class, struct, or interface to declare multiple methods with the same name, provided their signatures are unique within that class, struct, or interface.
- Overloading of instance constructors permits a class or struct to declare multiple instance constructors, provided their signatures are unique within that class or struct.
- Overloading of indexers permits a class, struct, or interface to declare multiple indexers, provided their signatures are unique within that class, struct, or interface.
- Overloading of operators permits a class or struct to declare multiple operators with the same name, provided their signatures are unique within that class or struct.

[*Example:* The following example shows a set of overloaded method declarations along with their signatures.

```

interface ITest
{
    void F();                // F()
    void F(int x);           // F(int)
    void F(ref int x);       // F(ref int)
    void F(out int x);       // F(out int)
    void F(int x, int y);    // F(int, int)
    int F(string s);         // F(string)
    int F(int x);            // F(int)      error
    void F(string[] a);      // F(string[])
    void F(params string[] a); // F(string[]) error
}

```

Note that any `ref` and `out` parameter modifiers (§17.5.1) are part of a signature. Thus, `F(int)`, `F(ref int)`, and `F(out int)` are all unique signatures. Also, note that the return type and the `params` modifier are not part of a signature, so it is not possible to overload solely based on return type or on the inclusion or exclusion of the `params` modifier. As such, the declarations of the methods `F(int)` and `F(params string[])` identified above, result in a compile-time error. *end example*

10.7 Scopes

The **scope** of a name is the region of program text within which it is possible to refer to the entity declared by the name without qualification of the name. Scopes can be **nested**, and an inner scope may redeclare the meaning of a name from an outer scope. [*Note:* This does not, however, remove the restriction imposed by §10.3 that within a nested block it is not possible to declare a local variable with the same name as a local variable in an enclosing block. *end note*] The name from the outer scope is then said to be **hidden** in the region of program text covered by the inner scope, and access to the outer name is only possible by qualifying the name.

- The scope of a namespace member declared by a *namespace-member-declaration* (§16.4) with no enclosing *namespace-declaration* is the entire program text.
- The scope of a namespace member declared by a *namespace-member-declaration* within a *namespace-declaration* whose fully qualified name is *N*, is the *namespace-body* of every *namespace-declaration* whose fully qualified name is *N* or starts with *N*, followed by a period.
- The scope of a name defined or imported by a *using-directive* (§16.3) extends over the *namespace-member-declarations* of the *compilation-unit* or *namespace-body* in which the *using-directive* occurs. A *using-directive* may make zero or more namespace or type names available within a particular *compilation-unit* or *namespace-body*, but does not contribute any new members to the underlying declaration space. In other words, a *using-directive* is not transitive, but, rather, affects only the *compilation-unit* or *namespace-body* in which it occurs.
- The scope of a member declared by a *class-member-declaration* (§17.2) is the *class-body* in which the declaration occurs. In addition, the scope of a class member extends to the *class-body* of those derived classes that are included in the accessibility domain (§10.5.2) of the member.
- The scope of a member declared by a *struct-member-declaration* (§18.2) is the *struct-body* in which the declaration occurs.
- The scope of a member declared by an *enum-member-declaration* (§21.3) is the *enum-body* in which the declaration occurs.
- The scope of a parameter declared in a *method-declaration* (§17.5) is the *method-body* of that *method-declaration*.
- The scope of a parameter declared in an *indexer-declaration* (§17.8) is the *accessor-declarations* of that *indexer-declaration*.
- The scope of a parameter declared in an *operator-declaration* (§17.9) is the *block* of that *operator-declaration*.
- The scope of a parameter declared in a *constructor-declaration* (§17.10) is the *constructor-initializer* and *block* of that *constructor-declaration*.
- The scope of a label declared in a *labeled-statement* (§15.4) is the *block* in which the declaration occurs.
- The scope of a local variable declared in a *local-variable-declaration* (§15.5.1) is the *block* in which the declaration occurs.
- The scope of a local variable declared in a *switch-block* of a `switch` statement (§15.7.2) is the *switch-block*.
- The scope of a local variable declared in a *for-initializer* of a `for` statement (§15.8.3) is the *for-initializer*, the *for-condition*, the *for-iterator*, and the contained *statement* of the `for` statement.
- The scope of a local constant declared in a *local-constant-declaration* (§15.5.2) is the *block* in which the declaration occurs. It is a compile-time error to refer to a local constant in a textual position that precedes its *constant-declarator*.

Within the scope of a namespace, class, struct, or enumeration member it is possible to refer to the member in a textual position that precedes the declaration of the member. [Example: For example

```
class A
{
    void F() {
        i = 1;
    }
    int i = 0;
}
```

Here, it is valid for `F` to refer to `i` before it is declared. end example]

Within the scope of a local variable, it is a compile-time error to refer to the local variable in a textual position that precedes the *local-variable-declarator* of the local variable. [Example: For example

```
class A
{
    int i = 0;
    void F() {
        i = 1;           // Error, use precedes declaration
        int i;
        i = 2;
    }
    void G() {
        int j = (j = 1); // valid
    }
    void H() {
        int a = 1, b = ++a; // valid
    }
}
```

In the F method above, the first assignment to *i* specifically does not refer to the field declared in the outer scope. Rather, it refers to the local variable and it results in a compile-time error because it textually precedes the declaration of the variable. In the G method, the use of *j* in the initializer for the declaration of *j* is valid because the use does not precede the *local-variable-declarator*. In the H method, a subsequent *local-variable-declarator* correctly refers to a local variable declared in an earlier *local-variable-declarator* within the same *local-variable-declaration*. end example]

[Note: The scoping rules for local variables are designed to guarantee that the meaning of a name used in an expression context is always the same within a block. If the scope of a local variable were to extend only from its declaration to the end of the block, then in the example above, the first assignment would assign to the instance variable and the second assignment would assign to the local variable, possibly leading to compile-time errors if the statements of the block were later to be rearranged.

The meaning of a name within a block may differ based on the context in which the name is used. In the example

```
using System;
class A {}

class Test
{
    static void Main() {
        string A = "hello, world";
        string s = A;           // expression context
        Type t = typeof(A);      // type context
        Console.WriteLine(s);    // writes "hello, world"
        Console.WriteLine(t.ToString()); // writes "Type: A"
    }
}
```

the name A is used in an expression context to refer to the local variable A and in a type context to refer to the class A. end note]

10.7.1 Name hiding

The scope of an entity typically encompasses more program text than the declaration space of the entity. In particular, the scope of an entity may include declarations that introduce new declaration spaces containing entities of the same name. Such declarations cause the original entity to become **hidden**. Conversely, an entity is said to be **visible** when it is not hidden.

Name hiding occurs when scopes overlap through nesting and when scopes overlap through inheritance. The characteristics of the two types of hiding are described in the following sections.

10.7.1.1 Hiding through nesting

Name hiding through nesting can occur as a result of nesting namespaces or types within namespaces, as a result of nesting types within classes or structs, and as a result of parameter and local variable declarations.

[*Example:* In the example

```
class A
{
    int i = 0;
    void F() {
        int i = 1;
    }
    void G() {
        i = 1;
    }
}
```

within the *F* method, the instance variable *i* is hidden by the local variable *i*, but within the *G* method, *i* still refers to the instance variable. *end example*]

When a name in an inner scope hides a name in an outer scope, it hides all overloaded occurrences of that name. [*Example:* In the example

```
class Outer
{
    static void F(int i) {}
    static void F(string s) {}
    class Inner
    {
        void G() {
            F(1);           // Invokes Outer.Inner.F
            F("Hello");     // Error
        }
        static void F(long l) {}
    }
}
```

the call *F*(1) invokes the *F* declared in *Inner* because all outer occurrences of *F* are hidden by the inner declaration. For the same reason, the call *F*("Hello") results in a compile-time error. *end example*]

10.7.1.2 Hiding through inheritance

Name hiding through inheritance occurs when classes or structs redeclare names that were inherited from base classes. This type of name hiding takes one of the following forms:

- A constant, field, property, event, or type introduced in a class or struct hides all base class members with the same name.
- A method introduced in a class or struct hides all non-method base class members with the same name, and all base class methods with the same signature (method name and parameter count, modifiers, and types).
- An indexer introduced in a class or struct hides all base class indexers with the same signature (parameter count and types).

The rules governing operator declarations (§17.9) make it impossible for a derived class to declare an operator with the same signature as an operator in a base class. Thus, operators never hide one another.

Contrary to hiding a name from an outer scope, hiding an accessible name from an inherited scope causes a warning to be reported. [*Example:* In the example

```
class Base
{
    public void F() {}
}
```

```

1      class Derived: Base
2      {
3          public void F() {}          // warning, hiding an inherited name
4      }

```

the declaration of `F` in `Derived` causes a warning to be reported. Hiding an inherited name is specifically not an error, since that would preclude separate evolution of base classes. For example, the above situation might have come about because a later version of `Base` introduced an `F` method that wasn't present in an earlier version of the class. Had the above situation been an error, then *any* change made to a base class in a separately versioned class library could potentially cause derived classes to become invalid. *end example*

The warning caused by hiding an inherited name can be eliminated through use of the `new` modifier:
[*Example:*

```

12     class Base
13     {
14         public void F() {}
15     }
16     class Derived: Base
17     {
18         new public void F() {}
19     }

```

The `new` modifier indicates that the `F` in `Derived` is “new”, and that it is indeed intended to hide the inherited member. *end example*

A declaration of a new member hides an inherited member only within the scope of the new member.
[*Example:*

```

24     class Base
25     {
26         public static void F() {}
27     }
28     class Derived: Base
29     {
30         new private static void F() {}    // Hides Base.F in Derived only
31     }
32     class MoreDerived: Derived
33     {
34         static void G() { F(); }          // Invokes Base.F
35     }

```

In the example above, the declaration of `F` in `Derived` hides the `F` that was inherited from `Base`, but since the new `F` in `Derived` has private access, its scope does not extend to `MoreDerived`. Thus, the call `F()` in `MoreDerived.G` is valid and will invoke `Base.F`. *end example*

10.8 Namespace and type names

Several contexts in a C# program require a *namespace-name* or a *type-name* to be specified. Either form of name is written as one or more identifiers separated by “.” tokens.

```

42     namespace-name:
43         namespace-or-type-name
44     type-name:
45         namespace-or-type-name
46     namespace-or-type-name:
47         identifier
48         namespace-or-type-name . identifier

```

A *type-name* is a *namespace-or-type-name* that refers to a type. Following resolution as described below, the *namespace-or-type-name* of a *type-name* must refer to a type, or otherwise a compile-time error occurs.

A *namespace-name* is a *namespace-or-type-name* that refers to a namespace. Following resolution as described below, the *namespace-or-type-name* of a *namespace-name* must refer to a namespace, or otherwise a compile-time error occurs.

The meaning of a *namespace-or-type-name* is determined as follows:

- If the *namespace-or-type-name* consists of a single identifier:
 - If the *namespace-or-type-name* appears within the body of a class or struct declaration, then starting with that class or struct declaration and continuing with each enclosing class or struct declaration (if any), if a member with the given name exists, is accessible, and denotes a type, then the *namespace-or-type-name* refers to that member. Note that non-type members (constants, fields, methods, properties, indexers, operators, instance constructors, destructors, and static constructors) are ignored when determining the meaning of a *namespace-or-type-name*.
 - Otherwise, starting with the namespace in which the *namespace-or-type-name* occurs, continuing with each enclosing namespace (if any), and ending with the global namespace, the following steps are evaluated until an entity is located:
 - If the namespace contains a namespace member with the given name, then the *namespace-or-type-name* refers to that member and, depending on the member, is classified as a namespace or a type.
 - Otherwise, if the namespace has a corresponding namespace declaration enclosing the location where the *namespace-or-type-name* occurs, then:
 - If the namespace declaration contains a using-alias-directive that associates the given name with an imported namespace or type, then the *namespace-or-type-name* refers to that namespace or type.
 - Otherwise, if the namespaces imported by the using-namespace-directives of the namespace declaration contain exactly one type with the given name, then the *namespace-or-type-name* refers to that type.
 - Otherwise, if the namespaces imported by the using-namespace-directives of the namespace declaration contain more than one type with the given name, then the *namespace-or-type-name* is ambiguous and an error occurs.
 - Otherwise, the *namespace-or-type-name* is undefined and a compile-time error occurs.
 - Otherwise, the *namespace-or-type-name* is of the form *N.I*, where *N* is a *namespace-or-type-name* consisting of all identifiers but the rightmost one, and *I* is the rightmost identifier. *N* is first resolved as a *namespace-or-type-name*. If the resolution of *N* is not successful, a compile-time error occurs. Otherwise, *N.I* is resolved as follows:
 - If *N* is a namespace and *I* is the name of an accessible member of that namespace, then *N.I* refers to that member and, depending on the member, is classified as a namespace or a type.
 - If *N* is a class or struct type and *I* is the name of an accessible type in *N*, then *N.I* refers to that type.
 - Otherwise, *N.I* is an *invalid namespace-or-type-name*, and a compile-time error occurs.

10.8.1 Fully qualified names

Every namespace and type has a **fully qualified name**, which uniquely identifies the namespace or type amongst all others. The fully qualified name of a namespace or type *N* is determined as follows:

- If N is a member of the global namespace, its fully qualified name is N .
- Otherwise, its fully qualified name is $S.N$, where S is the fully qualified name of the namespace or type in which N is declared.

In other words, the fully qualified name of N is the complete hierarchical path of identifiers that lead to N , starting from the global namespace. Because every member of a namespace or type must have a unique name, it follows that the fully qualified name of a namespace or type is always unique.

[*Example:* The example below shows several namespace and type declarations along with their associated fully qualified names.

```

class A {}           // A
namespace X          // X
{
    class B          // X.B
    {
        class C {}   // X.B.C
    }
    namespace Y      // X.Y
    {
        class D {}   // X.Y.D
    }
}
namespace X.Y        // X.Y
{
    class E {}       // X.Y.E
}

```

end example]

10.9 Automatic memory management

C# employs automatic memory management, which frees developers from manually allocating and freeing the memory occupied by objects. Automatic memory management policies are implemented by a garbage collector. The memory management life cycle of an object is as follows:

1. When the object is created, memory is allocated for it, the constructor is run, and the object is considered **live**.
2. If the object, or any part of it, cannot be accessed by any possible continuation of execution, other than the running of destructors, the object is considered **no longer in use**, and it becomes eligible for destruction. [*Note:* Implementations may choose to analyze code to determine which references to an object may be used in the future. For instance, if a local variable that is in scope is the only existing reference to an object, but that local variable is never referred to in any possible continuation of execution from the current execution point in the procedure, an implementation may (but is not required to) treat the object as no longer in use. *end note*]
3. Once the object is eligible for destruction, at some unspecified later time the destructor (§17.12) (if any) for the object is run. Unless overridden by explicit calls, the destructor for the object is run once only.
4. Once the destructor for an object is run, if that object, or any part of it, cannot be accessed by any possible continuation of execution, including the running of destructors, the object is considered **inaccessible** and the object becomes eligible for collection.
5. Finally, at some time after the object becomes eligible for collection, the garbage collector frees the memory associated with that object.

The garbage collector maintains information about object usage, and uses this information to make memory management decisions, such as where in memory to locate a newly created object, when to relocate an object, and when an object is no longer in use or inaccessible.

Like other languages that assume the existence of a garbage collector, C# is designed so that the garbage collector may implement a wide range of memory management policies. For instance, C# does not require that destructors be run or that objects be collected as soon as they are eligible, or that destructors be run in any particular order, or on any particular thread.

The behavior of the garbage collector can be controlled, to some degree, via static methods on the class `System.GC`. This class can be used to request a collection to occur, destructors to be run (or not run), and so forth.

[*Example:* Since the garbage collector is allowed wide latitude in deciding when to collect objects and run destructors, a conforming implementation may produce output that differs from that shown by the following code. The program

```
using System;
class A
{
    ~A() {
        Console.WriteLine("Destruct instance of A");
    }
}
class B
{
    object Ref;
    public B(object o) {
        Ref = o;
    }
    ~B() {
        Console.WriteLine("Destruct instance of B");
    }
}
class Test
{
    static void Main() {
        B b = new B(new A());
        b = null;
        GC.Collect();
        GC.WaitForPendingFinalizers();
    }
}
```

creates an instance of class A and an instance of class B. These objects become eligible for garbage collection when the variable `b` is assigned the value `null`, since after this time it is impossible for any user-written code to access them. The output could be either

```
Destruct instance of A
Destruct instance of B
```

or

```
Destruct instance of B
Destruct instance of A
```

because the language imposes no constraints on the order in which objects are garbage collected.

In subtle cases, the distinction between “eligible for destruction” and “eligible for collection” can be important. For example,

```
using System;
class A
{
    ~A() {
        Console.WriteLine("Destruct instance of A");
    }
    public void F() {
        Console.WriteLine("A.F");
        Test.RefA = this;
    }
}
```

```

1      class B
2      {
3          public A Ref;
4          ~B() {
5              Console.WriteLine("Destruct instance of B");
6              Ref.F();
7          }
8      }
9
10     class Test
11     {
12         public static A RefA;
13         public static B RefB;
14         static void Main() {
15             RefB = new B();
16             RefA = new A();
17             RefB.Ref = RefA;
18             RefB = null;
19             RefA = null;
20
21             // A and B now eligible for destruction
22             GC.Collect();
23             GC.WaitForPendingFinalizers();
24             // B now eligible for collection, but A is not
25             if (RefA != null)
26                 Console.WriteLine("RefA is not null");
27         }
28     }

```

29 In the above program, if the garbage collector chooses to run the destructor of B before the destructor of A, then the output of this program might be:

```

30     Destruct instance of A
31     Destruct instance of B
32     A.F
33     RefA is not null

```

34 Note that although the instance of A was not in use and A's destructor was run, it is still possible for methods of A (in this case, F) to be called from another destructor. Also, note that running of a destructor may cause an object to become usable from the mainline program again. In this case, the running of B's destructor caused an instance of A that was previously not in use to become accessible from the live reference RefA. After the call to `WaitForPendingFinalizers`, the instance of B is eligible for collection, but the instance of A is not, because of the reference RefA.

39 To avoid confusion and unexpected behavior, it is generally a good idea for destructors to only perform cleanup on data stored in their object's own fields, and not to perform any actions on referenced objects or static fields. *end example*

42 10.10 Execution order

43 Execution shall proceed such that the side effects of each executing thread are preserved at critical execution points. A **side effect** is defined as a read or write of a volatile field, a write to a non-volatile variable, a write to an external resource, and the throwing of an exception. The critical execution points at which the order of these side effects must be preserved are references to volatile fields (§17.4.3), `lock` statements (§15.12), and thread creation and termination. An implementation is free to change the order of execution of a C# program, subject to the following constraints:

- 49 • Data dependence is preserved within a thread of execution. That is, the value of each variable is computed as if all statements in the thread were executed in original program order.
- 51 • Initialization ordering rules are preserved (§17.4.4 and §17.4.5).
- 52 • The ordering of side effects is preserved with respect to volatile reads and writes (§17.4.3). Additionally, an implementation need not evaluate part of an expression if it can deduce that that expression's value is not used and that no needed side effects are produced (including any caused by calling a method or accessing a volatile field). When program execution is interrupted by an asynchronous event (such as an exception

- 1 thrown by another thread), it is not guaranteed that the observable side effects are visible in the original
- 2 program order.
- 3

11. Types

The types of the C# language are divided into two main categories: Value types and reference types.

type:

value-type

reference-type

A third category of types, pointers, is available only in unsafe code. This is discussed further in §25.2.

Value types differ from reference types in that variables of the value types directly contain their data, whereas variables of the reference types store **references** to their data, the latter being known as **objects**.

With reference types, it is possible for two variables to reference the same object, and thus possible for operations on one variable to affect the object referenced by the other variable. With value types, the variables each have their own copy of the data, and it is not possible for operations on one to affect the other.

C#'s type system is unified such that *a value of any type can be treated as an object*. Every type in C# directly or indirectly derives from the **object** class type, and **object** is the ultimate base class of all types. Values of reference types are treated as objects simply by viewing the values as type **object**. Values of value types are treated as objects by performing boxing and unboxing operations (§11.3).

11.1 Value types

A value type is either a struct type or an enumeration type. C# provides a set of predefined struct types called the **simple types**. The simple types are identified through reserved words.

value-type:

struct-type

enum-type

struct-type:

type-name

simple-type

simple-type:

numeric-type

bool

numeric-type:

integral-type

floating-point-type

decimal

integral-type:

sbyte

byte

short

ushort

int

uint

long

ulong

char

floating-point-type:

`float`

`double`

enum-type:

type-name

All value types implicitly inherit from class `object`. It is not possible for any type to derive from a value type, and value types are thus implicitly sealed (§17.1.1.2).

A variable of a value type always contains a value of that type. Unlike reference types, it is not possible for a value of a value type to be `null`, or to reference an object of a more derived type.

Assignment to a variable of a value type creates a *copy* of the value being assigned. This differs from assignment to a variable of a reference type, which copies the reference but not the object identified by the reference.

11.1.1 Default constructors

All value types implicitly declare a public parameterless instance constructor called the **default constructor**. The default constructor returns a zero-initialized instance known as the **default value** for the value type:

- For all *simple-types*, the default value is the value produced by a bit pattern of all zeros:
 - o For `sbyte`, `byte`, `short`, `ushort`, `int`, `uint`, `long`, and `ulong`, the default value is 0.
 - o For `char`, the default value is `'\x0000'`.
 - o For `float`, the default value is `0.0f`.
 - o For `double`, the default value is `0.0d`.
 - o For `decimal`, the default value is `0.0m`.
 - o For `bool`, the default value is `false`.
- For an *enum-type* `E`, the default value is 0.
- For a *struct-type*, the default value is the value produced by setting all value type fields to their default value and all reference type fields to `null`.

Like any other instance constructor, the default constructor of a value type is invoked using the `new` operator. [Note: For efficiency reasons, this requirement is not intended to actually have the implementation generate a constructor call. *end note*] In the example below, variables `i` and `j` are both initialized to zero.

```
class A
{
    void F() {
        int i = 0;
        int j = new int();
    }
}
```

Because every value type implicitly has a public parameterless instance constructor, it is not possible for a struct type to contain an explicit declaration of a parameterless constructor. A struct type is however permitted to declare parameterized instance constructors (§18.3.8).

11.1.2 Struct types

A struct type is a value type that can declare constants, fields, methods, properties, indexers, operators, instance constructors, static constructors, and nested types. Struct types are described in §18.

11.1.3 Simple types

C# provides a set of predefined struct types called the simple types. The simple types are identified through reserved words, but these reserved words are simply aliases for predefined struct types in the `System` namespace, as described in the table below.

Reserved word	Aliased type
<code>sbyte</code>	<code>System.SByte</code>
<code>byte</code>	<code>System.Byte</code>
<code>short</code>	<code>System.Int16</code>
<code>ushort</code>	<code>System.UInt16</code>
<code>int</code>	<code>System.Int32</code>
<code>uint</code>	<code>System.UInt32</code>
<code>long</code>	<code>System.Int64</code>
<code>ulong</code>	<code>System.UInt64</code>
<code>char</code>	<code>System.Char</code>
<code>float</code>	<code>System.Single</code>
<code>double</code>	<code>System.Double</code>
<code>bool</code>	<code>System.Boolean</code>
<code>decimal</code>	<code>System.Decimal</code>

Because a simple type aliases a struct type, every simple type has members. For example, `int` has the members declared in `System.Int32` and the members inherited from `System.Object`, and the following statements are permitted:

```
int i = int.MaxValue;           // System.Int32.MaxValue constant
string s = i.ToString();       // System.Int32.ToString() instance method
string t = 123.ToString();     // System.Int32.ToString() instance method
```

The simple types differ from other struct types in that they permit certain additional operations:

- Most simple types permit values to be created by writing *literals* (§9.4.4). For example, `123` is a literal of type `int` and `'a'` is a literal of type `char`. C# makes no provision for literals of struct types in general, and non-default values of other struct types are ultimately always created through instance constructors of those struct types.
- When the operands of an expression are all simple type constants, it is possible for the compiler to evaluate the expression at compile-time. Such an expression is known as a *constant-expression* (§14.15). Expressions involving operators defined by other struct types are not considered to be constant expressions.
- Through `const` declarations, it is possible to declare constants of the simple types (§17.3). It is not possible to have constants of other struct types, but a similar effect is provided by `static readonly` fields.
- Conversions involving simple types can participate in evaluation of conversion operators defined by other struct types, but a user-defined conversion operator can never participate in evaluation of another user-defined operator (§13.4.2).

11.1.4 Integral types

C# supports nine integral types: `sbyte`, `byte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, and `char`. The integral types have the following sizes and ranges of values:

- The `sbyte` type represents signed 8-bit integers with values between -128 and 127 .
 - The `byte` type represents unsigned 8-bit integers with values between 0 and 255 .
 - The `short` type represents signed 16-bit integers with values between -32768 and 32767 .
 - The `ushort` type represents unsigned 16-bit integers with values between 0 and 65535 .
 - The `int` type represents signed 32-bit integers with values between -2147483648 and 2147483647 .
 - The `uint` type represents unsigned 32-bit integers with values between 0 and 4294967295 .
 - The `long` type represents signed 64-bit integers with values between -9223372036854775808 and 9223372036854775807 .
 - The `ulong` type represents unsigned 64-bit integers with values between 0 and 18446744073709551615 .
 - The `char` type represents unsigned 16-bit integers with values between 0 and 65535 . The set of possible values for the `char` type corresponds to the Unicode character set. [*Note: Although `char` has the same representation as `ushort`, not all operations permitted on one type are permitted on the other. end note*]
- The integral-type unary and binary operators always operate with signed 32-bit precision, unsigned 32-bit precision, signed 64-bit precision, or unsigned 64-bit precision:
- For the unary `+` and `~` operators, the operand is converted to type `T`, where `T` is the first of `int`, `uint`, `long`, and `ulong` that can fully represent all possible values of the operand. The operation is then performed using the precision of type `T`, and the type of the result is `T`.
 - For the unary `-` operator, the operand is converted to type `T`, where `T` is the first of `int` and `long` that can fully represent all possible values of the operand. The operation is then performed using the precision of type `T`, and the type of the result is `T`. The unary `-` operator cannot be applied to operands of type `ulong`.
 - For the binary `+`, `-`, `*`, `/`, `%`, `&`, `^`, `|`, `==`, `!=`, `>`, `<`, `>=`, and `<=` operators, the operands are converted to type `T`, where `T` is the first of `int`, `uint`, `long`, and `ulong` that can fully represent all possible values of both operands. The operation is then performed using the precision of type `T`, and the type of the result is `T` (or `bool` for the relational operators). It is not permitted for one operand to be of type `long` and the other to be of type `ulong` with the binary operators.
 - For the binary `<<` and `>>` operators, the left operand is converted to type `T`, where `T` is the first of `int`, `uint`, `long`, and `ulong` that can fully represent all possible values of the operand. The operation is then performed using the precision of type `T`, and the type of the result is `T`.
- The `char` type is classified as an integral type, but it differs from the other integral types in two ways:
- There are no implicit conversions from other types to the `char` type. In particular, even though the `sbyte`, `byte`, and `ushort` types have ranges of values that are fully representable using the `char` type, implicit conversions from `sbyte`, `byte`, or `ushort` to `char` do not exist.
 - Constants of the `char` type must be written as *character-literals* or as *integer-literals* in combination with a cast to type `char`. For example, `(char)10` is the same as `'\x000A'`.
- The `checked` and `unchecked` operators and statements are used to control overflow checking for integral-type arithmetic operations and conversions (§14.5.12). In a `checked` context, an overflow produces a compile-time error or causes a `System.OverflowException` to be thrown. In an `unchecked` context, overflows are ignored and any high-order bits that do not fit in the destination type are discarded.

11.1.5 Floating point types

C# supports two floating-point types: `float` and `double`. The `float` and `double` types are represented using the 32-bit single-precision and 64-bit double-precision IEEE 754 formats, which provide the following sets of values:

- Positive zero and negative zero. In most situations, positive zero and negative zero behave identically as the simple value zero, but certain operations distinguish between the two (§14.7.2).
- Positive infinity and negative infinity. Infinities are produced by such operations as dividing a non-zero number by zero. For example, `1.0 / 0.0` yields positive infinity, and `-1.0 / 0.0` yields negative infinity.
- The **Not-a-Number** value, often abbreviated NaN. NaNs are produced by invalid floating-point operations, such as dividing zero by zero.
- The finite set of non-zero values of the form $s \times m \times 2^e$, where s is 1 or -1 , and m and e are determined by the particular floating-point type: For `float`, $0 < m < 2^{24}$ and $-149 \leq e \leq 104$, and for `double`, $0 < m < 2^{53}$ and $-1075 \leq e \leq 970$. Denormalized floating-point numbers are considered valid non-zero values.

The `float` type can represent values ranging from approximately 1.5×10^{-45} to 3.4×10^{38} with a precision of 7 digits.

The `double` type can represent values ranging from approximately 5.0×10^{-324} to 1.7×10^{308} with a precision of 15–16 digits.

If one of the operands of a binary operator is of a floating-point type, then the other operand must be of an integral type or a floating-point type, and the operation is evaluated as follows:

- If one of the operands is of an integral type, then that operand is converted to the floating-point type of the other operand.
- Then, if either of the operands is of type `double`, the other operand is converted to `double`, the operation is performed using at least `double` range and precision, and the type of the result is `double` (or `bool` for the relational operators).
- Otherwise, the operation is performed using at least `float` range and precision, and the type of the result is `float` (or `bool` for the relational operators).

The floating-point operators, including the assignment operators, never produce exceptions. Instead, in exceptional situations, floating-point operations produce zero, infinity, or NaN, as described below:

- If the result of a floating-point operation is too small for the destination format, the result of the operation becomes positive zero or negative zero.
- If the result of a floating-point operation is too large for the destination format, the result of the operation becomes positive infinity or negative infinity.
- If a floating-point operation is invalid, the result of the operation becomes NaN.
- If one or both operands of a floating-point operation is NaN, the result of the operation becomes NaN.

Floating-point operations may be performed with higher precision than the result type of the operation. For example, some hardware architectures support an “extended” or “long double” floating-point type with greater range and precision than the `double` type, and implicitly perform all floating-point operations using this higher precision type. Only at excessive cost in performance can such hardware architectures be made to perform floating-point operations with *less* precision, and rather than require an implementation to forfeit both performance and precision, C# allows a higher precision type to be used for all floating-point operations. Other than delivering more precise results, this rarely has any measurable effects. However, in expressions of the form `x * y / z`, where the multiplication produces a result that is outside the `double` range, but the subsequent division brings the temporary result back into the `double` range, the fact that the expression is evaluated in a higher range format may cause a finite result to be produced instead of an infinity.

11.1.6 The decimal type

The `decimal` type is a 128-bit data type suitable for financial and monetary calculations. The `decimal` type can represent values ranging from 1.0×10^{-28} to approximately 7.9×10^{28} with 28–29 significant digits.

The finite set of values of type `decimal` are of the form $-1^s \times c \times 10^{-e}$, where the sign s is 0 or 1, the coefficient c is given by $0 \leq c < 2^{96}$, and the scale e is such that $0 \leq e \leq 28$. The `decimal` type does not support signed zeros, infinities, or NaN's.

A `decimal` is represented as a 96-bit integer scaled by a power of ten. For `decimals` with an absolute value less than 1.0m, the value is exact to the 28th decimal place, but no further. For `decimals` with an absolute value greater than or equal to 1.0m, the value is exact to 28 or 29 digits. Contrary to the `float` and `double` data types, decimal fractional numbers such as 0.1 can be represented exactly in the `decimal` representation. In the `float` and `double` representations, such numbers are often infinite fractions, making those representations more prone to round-off errors.

If one of the operands of a binary operator is of type `decimal`, then the other operand must be of an integral type or of type `decimal`. If an integral type operand is present, it is converted to `decimal` before the operation is performed.

The result of an operation on values of type `decimal` is that which would result from calculating an exact result (preserving scale, as defined for each operator) and then rounding to fit the representation. Results are rounded to the nearest representable value, and, when a result is equally close to two representable values, to the value that has an even number in the least significant digit position (this is known as “banker’s rounding”). That is, results are exact to 28 or 29 digits, but to no more than 28 decimal places. A zero result always has a sign of 0 and a scale of 0.

If a decimal arithmetic operation produces a value that is too small for the decimal format after rounding, the result of the operation becomes zero. If a `decimal` arithmetic operation produces a result that is too large for the `decimal` format, a `System.OverflowException` is thrown.

The `decimal` type has greater precision but smaller range than the floating-point types. Thus, conversions from the floating-point types to `decimal` might produce overflow exceptions, and conversions from `decimal` to the floating-point types might cause loss of precision. For these reasons, no implicit conversions exist between the floating-point types and `decimal`, and without explicit casts, it is not possible to mix floating-point and `decimal` operands in the same expression.

11.1.7 The `bool` type

The `bool` type represents boolean logical quantities. The possible values of type `bool` are `true` and `false`.

No standard conversions exist between `bool` and other types. In particular, the `bool` type is distinct and separate from the integral types, and a `bool` value cannot be used in place of an integral value, and vice versa.

[*Note:* In the C and C++ languages, a zero integral or floating-point value, or a null pointer can be converted to the boolean value `false`, and a non-zero integral or floating-point value, or a non-null pointer can be converted to the boolean value `true`. In C#, such conversions are accomplished by explicitly comparing an integral or floating-point value to zero, or by explicitly comparing an object reference to `null`. *end note*]

11.1.8 Enumeration types

An enumeration type is a distinct type with named constants. Every enumeration type has an underlying type, which must be `byte`, `sbyte`, `short`, `ushort`, `int`, `uint`, `long` or `ulong`. Enumeration types are defined through enumeration declarations (§21.1).

11.2 Reference types

A reference type is a class type, an interface type, an array type, or a delegate type.

reference-type:

class-type

interface-type

array-type

delegate-type

```

1      class-type:
2          type-name
3          object
4          string
5
6      interface-type:
7          type-name
8
9      array-type:
10         non-array-type rank-specifiers
11
12     non-array-type:
13         type
14
15     rank-specifiers:
16         rank-specifier
17         rank-specifiers rank-specifier
18
19     rank-specifier:
20         [ dim-separatorsopt ]
21
22     dim-separators:
23         ,
24         dim-separators ,
25
26     delegate-type:
27         type-name

```

A reference type value is a reference to an **instance** of the type, the latter known as an **object**. The special value `null` is compatible with all reference types and indicates the absence of an instance.

11.2.1 Class types

A class type defines a data structure that contains data members (constants and fields), function members (methods, properties, events, indexers, operators, instance constructors, destructors, and static constructors), and nested types. Class types support inheritance, a mechanism whereby derived classes can extend and specialize base classes. Instances of class types are created using *object-creation-expressions* (§14.5.10.1).

Class types are described in §17.

11.2.2 The object type

The `object` class type is the ultimate base class of all other types. Every type in C# directly or indirectly derives from the `object` class type.

The keyword `object` is simply an alias for the predefined class `System.Object`.

11.2.3 The string type

The `string` type is a sealed class type that inherits directly from `object`. Instances of the `string` class represent Unicode character strings.

Values of the `string` type can be written as string literals (§9.4.4).

The keyword `string` is simply an alias for the predefined class `System.String`.

11.2.4 Interface types

An interface defines a contract. A class or struct that implements an interface must adhere to its contract. An interface may inherit from multiple base interfaces, and a class or struct may implement multiple interfaces.

Interface types are described in §20.

11.2.5 Array types

An array is a data structure that contains zero or more variables which are accessed through computed indices. The variables contained in an array, also called the elements of the array, are all of the same type, and this type is called the element type of the array.

Array types are described in §19.

11.2.6 Delegate types

A delegate is a data structure that refers to one or more methods, and for instance methods, it also refers to their corresponding object instances.

[*Note:* The closest equivalent of a delegate in C or C++ is a function pointer, but whereas a function pointer can only reference static functions, a delegate can reference both static and instance methods. In the latter case, the delegate stores not only a reference to the method's entry point, but also a reference to the object instance on which to invoke the method. *end note*]

Delegate types are described in §22.

11.3 Boxing and unboxing

The concept of boxing and unboxing is central to C#'s type system. It provides a bridge between *value-types* and *reference-types* by permitting any value of a *value-type* to be converted to and from type **object**. Boxing and unboxing enables a unified view of the type system wherein a value of any type can ultimately be treated as an object.

11.3.1 Boxing conversions

A boxing conversion permits any *value-type* to be implicitly converted to the type **object** or to any *interface-type* implemented by the *value-type*. Boxing a value of a *value-type* consists of allocating an object instance and copying the *value-type* value into that instance.

The actual process of boxing a value of a *value-type* is best explained by imagining the existence of a **boxing class** for that type. [*Example:* For any *value-type* *T*, the boxing class behaves as if it were declared as follows:

```
sealed class T_Box
{
    T value;
    public T_Box(T t) {
        value = t;
    }
}
```

Boxing of a value *v* of type *T* now consists of executing the expression `new T_Box(v)`, and returning the resulting instance as a value of type **object**. Thus, the statements

```
int i = 123;
object box = i;
```

conceptually correspond to

```
int i = 123;
object box = new int_Box(i);
```

end example]

Boxing classes like `T_Box` and `int_Box` above don't actually exist and the dynamic type of a boxed value isn't actually a class type. Instead, a boxed value of type *T* has the dynamic type *T*, and a dynamic type check using the `is` operator can simply reference type *T*. [*Example:* For example,

```

1      int i = 123;
2      object box = i;
3      if (box is int) {
4          Console.WriteLine("Box contains an int");
5      }

```

6 will output the string “Box contains an int” on the console. *end example*]

7 A boxing conversion implies *making a copy* of the value being boxed. This is different from a conversion of
8 a *reference-type* to type **object**, in which the value continues to reference the same instance and simply is
9 regarded as the less derived type **object**. [Example: For example, given the declaration

```

10     struct Point
11     {
12         public int x, y;
13         public Point(int x, int y) {
14             this.x = x;
15             this.y = y;
16         }
17     }

```

18 the following statements

```

19     Point p = new Point(10, 10);
20     object box = p;
21     p.x = 20;
22     Console.WriteLine(((Point)box).x);

```

23 will output the value 10 on the console because the implicit boxing operation that occurs in the assignment
24 of **p** to **box** causes the value of **p** to be copied. Had **Point** been declared a **class** instead, the value 20
25 would be output because **p** and **box** would reference the same instance. *end example*]

26 11.3.2 Unboxing conversions

27 An unboxing conversion permits an explicit conversion from type **object** to any *value-type* or from any
28 *interface-type* to any *value-type* that implements the *interface-type*. An unboxing operation consists of first
29 checking that the object instance is a boxed value of the given *value-type*, and then copying the value out of
30 the instance.

31 Referring to the imaginary boxing class described in the previous section, an unboxing conversion of an
32 object **box** to a *value-type* **T** consists of executing the expression `((T_Box)box).value`. [Example: Thus,
33 the statements

```

34     object box = 123;
35     int i = (int)box;

```

36 conceptually correspond to

```

37     object box = new int_Box(123);
38     int i = ((int_Box)box).value;

```

39 *end example*]

40 For an unboxing conversion to a given *value-type* to succeed at run-time, the value of the source operand
41 must be a reference to an object that was previously created by boxing a value of that *value-type*. If the
42 source operand is **null** or a reference to an incompatible object, a **System.InvalidCastException** is
43 thrown.

12. Variables

Variables represent storage locations. Every variable has a type that determines what values can be stored in the variable. C# is a type-safe language, and the C# compiler guarantees that values stored in variables are always of the appropriate type. The value of a variable can be changed through assignment or through use of the ++ and -- operators.

A variable must be **definitely assigned** (§12.3) before its value can be obtained.

As described in the following sections, variables are either **initially assigned** or **initially unassigned**. An initially assigned variable has a well-defined initial value and is always considered definitely assigned. An initially unassigned variable has no initial value. For an initially unassigned variable to be considered definitely assigned at a certain location, an assignment to the variable must occur in every possible execution path leading to that location.

12.1 Variable categories

C# defines seven categories of variables: static variables, instance variables, array elements, value parameters, reference parameters, output parameters, and local variables. The sections that follow describe each of these categories.

[*Example:* In the example

```
class A
{
    public static int x;
    int y;

    void F(int[] v, int a, ref int b, out int c) {
        int i = 1;
        c = a + b++;
    }
}
```

x is a static variable, y is an instance variable, v[0] is an array element, a is a value parameter, b is a reference parameter, c is an output parameter, and i is a local variable. *end example*]

12.1.1 Static variables

A field declared with the `static` modifier is called a **static variable**. A static variable comes into existence before execution of the static constructor (§17.11) for its containing type, and ceases to exist when the associated application domain ceases to exist..

The initial value of a static variable is the default value (§12.2) of the variable's type.

For the purposes of definite assignment checking, a static variable is considered initially assigned.

12.1.2 Instance variables

A field declared without the `static` modifier is called an **instance variable**.

12.1.2.1 Instance variables in classes

An instance variable of a class comes into existence when a new instance of that class is created, and ceases to exist when there are no references to that instance and the instance's destructor (if any) has executed.

The initial value of an instance variable of a class is the default value (§12.2) of the variable's type.

For the purpose of definite assignment checking, an instance variable is considered initially assigned.

12.1.2.2 Instance variables in structs

An instance variable of a struct has exactly the same lifetime as the struct variable to which it belongs. In other words, when a variable of a struct type comes into existence or ceases to exist, so too do the instance variables of the struct.

The initial assignment state of an instance variable of a struct is the same as that of the containing struct variable. In other words, when a struct variable is considered initially assigned, so too are its instance variables, and when a struct variable is considered initially unassigned, its instance variables are likewise unassigned.

12.1.3 Array elements

The elements of an array come into existence when an array instance is created, and cease to exist when there are no references to that array instance.

The initial value of each of the elements of an array is the default value (§12.2) of the type of the array elements.

For the purpose of definite assignment checking, an array element is considered initially assigned.

12.1.4 Value parameters

A parameter declared without a `ref` or `out` modifier is a *value parameter*.

A value parameter comes into existence upon invocation of the function member (method, instance constructor, accessor, or operator) to which the parameter belongs, and is initialized with the value of the argument given in the invocation. A value parameter ceases to exist upon return of the function member.

For the purpose of definite assignment checking, a value parameter is considered initially assigned.

12.1.5 Reference parameters

A parameter declared with a `ref` modifier is a *reference parameter*.

A reference parameter does not create a new storage location. Instead, a reference parameter represents the same storage location as the variable given as the argument in the function member invocation. Thus, the value of a reference parameter is always the same as the underlying variable.

The following definite assignment rules apply to reference parameters. Note the different rules for output parameters described in §12.1.6.

- A variable must be definitely assigned (§12.3) before it can be passed as a reference parameter in a function member invocation.

- Within a function member, a reference parameter is considered initially assigned.

Within an instance method or instance accessor of a struct type, the `this` keyword behaves exactly as a reference parameter of the struct type (§14.5.7).

12.1.6 Output parameters

A parameter declared with an `out` modifier is an *output parameter*.

An output parameter does not create a new storage location. Instead, an output parameter represents the same storage location as the variable given as the argument in the function member invocation. Thus, the value of an output parameter is always the same as the underlying variable.

The following definite assignment rules apply to output parameters. Note the different rules for reference parameters described in §12.1.5.

- A variable need not be definitely assigned before it can be passed as an output parameter in a function member invocation.
- Following the normal completion of a function member invocation, each variable that was passed as an output parameter is considered assigned in that execution path.
- Within a function member, an output parameter is considered initially unassigned.
- Every output parameter of a function member must be definitely assigned (§12.3) before the function member returns normally.

Within an instance constructor of a struct type, the `this` keyword behaves exactly as an output parameter of the struct type (§14.5.7).

12.1.7 Local variables

A **local variable** is declared by a *local-variable-declaration*, which may occur in a *block*, a *for-statement*, a *switch-statement*, or a *using-statement*.

The lifetime of a local variable is the portion of program execution during which storage is guaranteed to be reserved for it. This lifetime extends from entry into the *block*, *for-statement*, *switch-statement*, or *using-statement* with which it is associated, until execution of that *block*, *for-statement*, *switch-statement*, or *using-statement* ends in any way. (Entering an enclosed *block* or calling a method suspends, but does not end, execution of the current *block*, *for-statement*, *switch-statement*, or *using-statement*.) If the parent *block*, *for-statement*, *switch-statement*, or *using-statement* is entered recursively, a new instance of the local variable is created each time, and its *local-variable-initializer*, if any, is evaluated each time.

A local variable is not automatically initialized and thus has no default value. For the purpose of definite assignment checking, a local variable is considered initially unassigned. A *local-variable-declaration* may include a *local-variable-initializer*, in which case the variable is considered definitely assigned in its entire scope, except within the expression provided in the *local-variable-initializer*.

Within the scope of a local variable, it is a compile-time error to refer to that local variable in a textual position that precedes its *local-variable-declarator*.

[*Note:* The actual lifetime of a local variable is implementation-dependent. For example, a compiler might statically determine that a local variable in a block is only used for a small portion of that block. Using this analysis, the compiler could generate code that results in the variable's storage having a shorter lifetime than its containing block.

The storage referred to by a local reference variable is reclaimed independently of the lifetime of that local reference variable (§10.9). *end note*]

A local variable is also declared by a *foreach-statement* and by a *specific-catch-clause* for a *try-statement*. For a *foreach-statement*, the local variable is an iteration variable (§15.8.4). For a *specific-catch-clause*, the local variable is an exception variable (§15.10). A local variable declared by a *foreach-statement* or *specific-catch-clause* is considered definitely assigned in its entire scope.

12.2 Default values

The following categories of variables are automatically initialized to their default values:

- Static variables.
- Instance variables of class instances.
- Array elements.

The default value of a variable depends on the type of the variable and is determined as follows:

- For a variable of a *value-type*, the default value is the same as the value computed by the *value-type*'s default constructor (§11.1.1).
 - For a variable of a *reference-type*, the default value is `null`.
- [*Note*: Initialization to default values is typically done by having the memory manager or garbage collector initialize memory to all-bits-zero before it is allocated for use. For this reason, it is convenient to use all-bits-zero to represent the null reference. *end note*]

12.3 Definite assignment

At a given location in the executable code of a function member, a variable is said to be **definitely assigned** if the compiler can prove, by static flow analysis, that the variable has been automatically initialized or has been the target of at least one assignment. The rules of definite assignment are:

- An initially assigned variable (§12.3.1) is always considered definitely assigned.
- An initially unassigned variable (§12.3.2) is considered definitely assigned at a given location if all possible execution paths leading to that location contain at least one of the following:
 - A simple assignment (§14.13.1) in which the variable is the left operand.
 - An invocation expression (§14.5.5) or object creation expression (§14.5.10.1) that passes the variable as an output parameter.
 - For a local variable, a local variable declaration (§15.5) that includes a variable initializer.

The definite assignment states of instance variables of a *struct-type* variable are tracked individually as well as collectively. In addition to the rules above, the following rules apply to *struct-type* variables and their instance variables:

- An instance variable is considered definitely assigned if its containing *struct-type* variable is considered definitely assigned.
- A *struct-type* variable is considered definitely assigned if each of its instance variables is considered definitely assigned.

Definite assignment is a requirement in the following contexts:

- A variable must be definitely assigned at each location where its value is obtained. [*Note*: This ensures that undefined values never occur. *end note*] The occurrence of a variable in an expression is considered to obtain the value of the variable, except when
 - the variable is the left operand of a simple assignment,
 - the variable is passed as an output parameter, or
 - the variable is a *struct-type* variable and occurs as the left operand of a member access.
- A variable must be definitely assigned at each location where it is passed as a reference parameter. [*Note*: This ensures that the function member being invoked can consider the reference parameter initially assigned. *end note*]
- All output parameters of a function member must be definitely assigned at each location where the function member returns (through a `return` statement or through execution reaching the end of the function member body). [*Note*: This ensures that function members do not return undefined values in output parameters, thus enabling the compiler to consider a function member invocation that takes a variable as an output parameter equivalent to an assignment to the variable. *end note*]
- The `this` variable of a *struct-type* instance constructor must be definitely assigned at each location where that instance constructor returns.

12.3.1 Initially assigned variables

The following categories of variables are classified as initially assigned:

- 1 • Static variables.
- 2 • Instance variables of class instances.
- 3 • Instance variables of initially assigned struct variables.
- 4 • Array elements.
- 5 • Value parameters.
- 6 • Reference parameters.
- 7 • Variables declared in a `catch` clause or a `foreach` statement.

8 12.3.2 Initially unassigned variables

9 The following categories of variables are classified as initially unassigned:

- 10 • Instance variables of initially unassigned struct variables.
- 11 • Output parameters, including the `this` variable of struct instance constructors.
- 12 • Local variables, except those declared in a `catch` clause or a `foreach` statement.

13 12.3.3 Precise rules for determining definite assignment

14 In order to determine that each used variable is definitely assigned, the compiler must use a process that is
15 equivalent to the one described in this section.

16 The compiler processes the body of each function member that has one or more initially unassigned
17 variables. For each initially unassigned variable *v*, the compiler determines a *definite assignment state* for *v*
18 at each of the following points in the function member:

- 19 • At the beginning of each statement
- 20 • At the end point (§15.1) of each statement
- 21 • On each arc which transfers control to another statement or to the end point of a statement
- 22 • At the beginning of each expression
- 23 • At the end of each expression

24 The definite assignment state of *v* can be either:

- 25 • Definitely assigned. This indicates that on all possible control flows to this point, *v* has been
26 assigned a value.
- 27 • Not definitely assigned. For the state of a variable at the end of an expression of type `bool`, the state
28 of a variable the isn't definitely assigned may (but doesn't necessarily) fall into one of the following
29 sub-states:
 - 30 ○ Definitely assigned after true expression. This state indicates that *v* is definitely assigned if
31 the boolean expression evaluated as true, but is not necessarily assigned if the boolean
32 expression evaluated as false.
 - 33 ○ Definitely assigned after false expression. This state indicates that *v* is definitely assigned if
34 the boolean expression evaluated as false, but is not necessarily assigned if the boolean
35 expression evaluated as true.

36 The following rules govern how the state of a variable *v* is determined at each location.

37 12.3.3.1 General rules for statements

- 38 • *v* is not definitely assigned at the beginning of a function member body.
- 39 • *v* is definitely assigned at the beginning of any unreachable statement.

- The definite assignment state of *v* at the beginning of any other statement is determined by checking the definite assignment state of *v* on all control flow transfers that target the beginning of that statement. If (and only if) *v* is definitely assigned on all such control flow transfers, then *v* is definitely assigned at the beginning of the statement. The set of possible control flow transfers is determined in the same way as for checking statement reachability (§15.1).
- The definite assignment state of *v* at the end point of a block, **checked**, **unchecked**, **if**, **while**, **do**, **for**, **foreach**, **lock**, **using**, or **switch** statement is determined by checking the definite assignment state of *v* on all control flow transfers that target the end point of that statement. If *v* is definitely assigned on all such control flow transfers, then *v* is definitely assigned at the end point of the statement. Otherwise, *v* is not definitely assigned at the end point of the statement. The set of possible control flow transfers is determined in the same way as for checking statement reachability (§15.1).

12.3.3.2 Block statements, **checked**, and **unchecked** statements

The definite assignment state of *v* on the control transfer to the first statement of the statement list in the block (or to the end point of the block, if the statement list is empty) is the same as the definite assignment statement of *v* before the block, **checked**, or **unchecked** statement.

12.3.3.3 Expression statements

For an expression statement *stmt* that consists of the expression *expr*:

- *v* has the same definite assignment state at the beginning of *expr* as at the beginning of *stmt*.
- If *v* is definitely assigned at the end of *expr*, it is definitely assigned at the end point of *stmt*; otherwise, it is not definitely assigned at the end point of *stmt*.

12.3.3.4 Declaration statements

- If *stmt* is a declaration statement without initializers, then *v* has the same definite assignment state at the end point of *stmt* as at the beginning of *stmt*.
- If *stmt* is a declaration statement with initializers, then the definite assignment state for *v* is determined as if *stmt* were a statement list, with one assignment statement for each declaration with an initializer (in the order of declaration).

12.3.3.5 If statements

For an **if** statement *stmt* of the form:

if (*expr*) *then-stmt* **else** *else-stmt*

- *v* has the same definite assignment state at the beginning of *expr* as at the beginning of *stmt*.
- If *v* is definitely assigned at the end of *expr*, then it is definitely assigned on the control flow transfer to *then-stmt* and to either *else-stmt* or to the end-point of *stmt* if there is no else clause.
- If *v* has the state “definitely assigned after true expression” at the end of *expr*, then it is definitely assigned on the control flow transfer to *then-stmt*, and not definitely assigned on the control flow transfer to either *else-stmt* or to the end-point of *stmt* if there is no else clause.
- If *v* has the state “definitely assigned after false expression” at the end of *expr*, then it is definitely assigned on the control flow transfer to *else-stmt*, and not definitely assigned on the control flow transfer to *then-stmt*. It is definitely assigned at the end-point of *stmt* if and only if it is definitely assigned at the end-point of *then-stmt*.
- Otherwise, *v* is considered not definitely assigned on the control flow transfer to either the *then-stmt* or *else-stmt*, or to the end-point of *stmt* if there is no else clause.

1 12.3.3.6 Switch statements

2 In a `switch` statement *stmt* with controlling expression *expr*:

- 3 • The definite assignment state of *v* at the beginning of *expr* is the same as the state of *v* at the
- 4 beginning of *stmt*.
- 5 • The definite assignment state of *v* on the control flow transfer to a reachable switch block statement
- 6 list is the same as the definite assignment state of *v* at the end of *expr*.

7 12.3.3.7 While statements

8 For a `while` statement *stmt* of the form:9 `while (expr) while-body`

- 10 • *v* has the same definite assignment state at the beginning of *expr* as at the beginning of *stmt*.
- 11 • If *v* is definitely assigned at the end of *expr*, then it is definitely assigned on the control flow transfer
- 12 to *while-body* and to the end point of *stmt*.
- 13 • If *v* has the state “definitely assigned after true expression” at the end of *expr*, then it is definitely
- 14 assigned on the control flow transfer to *while-body*, but not definitely assigned at the end-point of
- 15 *stmt*.
- 16 • If *v* has the state “definitely assigned after false expression” at the end of *expr*, then it is definitely
- 17 assigned on the control flow transfer to the end point of *stmt*.

18 12.3.3.8 Do statements

19 For a `do` statement *stmt* of the form:20 `do do-body while (expr);`

- 21 • *v* has the same definite assignment state on the control flow transfer from the beginning of *stmt* to
- 22 *do-body* as at the beginning of *stmt*.
- 23 • *v* has the same definite assignment state at the beginning of *expr* as at the end point of *do-body*.
- 24 • If *v* is definitely assigned at the end of *expr*, then it is definitely assigned on the end point of *stmt*.
- 25 • If *v* has the state “definitely assigned after false expression” at the end of *expr*, then it is definitely
- 26 assigned on the control flow transfer to the end point of *stmt*.

27 12.3.3.9 For statements

28 Definite assignment checking for a `for` statement of the form:29 `for (for-initializer; for-condition; for-iterator) embedded-statement`

30 is done as if the statement were written:

```

31 {
32     for-initializer;
33     while (for-condition) {
34         embedded-statement;
35         for-iterator;
36     }
37 }
```

38 If the *for-condition* is omitted from the `for` statement, then evaluation of definite assignment proceeds as if

39 *for-condition* were replaced with `true` in the above expansion.

12.3.3.10 Break, continue, and goto statements

The definite assignment state of *v* on the control flow transfer caused by a **break**, **continue**, or **goto** statement is the same as the definite assignment state of *v* at the beginning of the statement.

12.3.3.11 Throw statements

For a statement *stmt* of the form

```
throw expr ;
```

The definite assignment state of *v* at the beginning of *expr* is the same as the definite assignment state of *v* at the beginning of *stmt*.

12.3.3.12 Return statements

For a statement *stmt* of the form

```
return expr ;
```

- The definite assignment state of *v* at the beginning of *expr* is the same as the definite assignment state of *v* at the beginning of *stmt*.
- If *v* is an output parameter, then it must be definitely assigned either:
 - after *expr*
 - or at the end of the **finally** block of a **try-finally** or **try-catch-finally** that encloses the **return** statement.

12.3.3.13 Try-catch statements

For a statement *stmt* of the form:

```
try try-block
catch(...) catch-block-1
...
catch(...) catch-block-n
```

- The definite assignment state of *v* at the beginning of *try-block* is the same as the definite assignment state of *v* at the beginning of *stmt*.
- The definite assignment state of *v* at the beginning of *catch-block-i* (for any *i*) is the same as the definite assignment state of *v* at the beginning of *stmt*.
- The definite assignment state of *v* at the end-point of *stmt* is definitely assigned if (and only if) *v* is definitely assigned at the end-point of *try-block* and every *catch-block-i* (for every *i* from 1 to *n*).

12.3.3.14 Try-finally statements

For a **try** statement *stmt* of the form:

```
try try-block finally finally-block
```

- The definite assignment state of *v* at the beginning of *try-block* is the same as the definite assignment state of *v* at the beginning of *stmt*.
- The definite assignment state of *v* at the beginning of *finally-block* is the same as the definite assignment state of *v* at the beginning of *stmt*.
- The definite assignment state of *v* at the end-point of *stmt* is definitely assigned if (and only if) either:
 - *v* is definitely assigned at the end-point of *try-block*
 - *v* is definitely assigned at the end-point of *finally-block*

If a control flow transfer (for example, a `goto` statement) is made that begins within *try-block*, and ends outside of *try-block*, then *v* is also considered definitely assigned on that control flow transfer if *v* is definitely assigned at the end-point of *finally-block*. (This is not an only if—if *v* is definitely assigned for another reason on this control flow transfer, then it is still considered definitely assigned.)

12.3.3.15 Try-catch-finally statements

Definite assignment analysis for a `try-catch-finally` statement of the form:

```
try try-block
catch(...) catch-block-1
...
catch(...) catch-block-n
finally finally-block
```

is done as if the statement were a `try-finally` statement enclosing a `try-catch` statement:

```
try {
    try try-block
    catch(...) catch-block-1
    ...
    catch(...) catch-block-n
}
finally finally-block
```

[Example: The following example demonstrates how the different blocks of a `try` statement (§15.10) affect definite assignment.

```
class A
{
    static void F() {
        int i, j;
        try {
            goto LABEL:
            // neither i nor j definitely assigned
            i = 1;
            // i definitely assigned
        }
        catch {
            // neither i nor j definitely assigned
            i = 3;
            // i definitely assigned
        }
        finally {
            // neither i nor j definitely assigned
            j = 5;
            // j definitely assigned
        }
        // i and j definitely assigned
    LABEL:
        // j definitely assigned
    }
}
```

end example]

12.3.3.16 Foreach statements

For a `foreach` statement *stmt* of the form:

```
foreach (type identifier in expr) embedded-statement
```

- The definite assignment state of *v* at the beginning of *expr* is the same as the state of *v* at the beginning of *stmt*.

- The definite assignment state of v on the control flow transfer to *embedded-statement* or to the end point of *stmt* is the same as the state of v at the end of *expr*.

12.3.3.17 Using statements

For a **using** statement *stmt* of the form:

using (*resource-acquisition*) *embedded-statement*

- The definite assignment state of v at the beginning of *resource-acquisition* is the same as the state of v at the beginning of *stmt*.
- The definite assignment state of v on the control flow transfer to *embedded-statement* is the same as the state of v at the end of *resource-acquisition*.

12.3.3.18 Lock statements

For a **lock** statement *stmt* of the form:

lock (*expr*) *embedded-statement*

- The definite assignment state of v at the beginning of *expr* is the same as the state of v at the beginning of *stmt*.
- The definite assignment state of v on the control flow transfer to *embedded-statement* is the same as the state of v at the end of *expr*.

12.3.3.19 General rules for simple expressions

The following rule applies to these kinds of expressions: literals (§14.5.1), simple names (§14.5.2), member access expressions (§14.5.4), non-indexed base access expressions (§14.5.8), and **typeof** expressions (§14.5.11).

- The definite assignment state of v at the end of such an expression is the same as the definite assignment state of v at the beginning of the expression.

12.3.3.20 General rules for expressions with embedded expressions

The following rules apply to these kinds of expressions: parenthesized expressions (§14.5.3), element access expressions (§14.5.6), base access expressions with indexing (§14.5.8), increment and decrement expressions (§14.5.9, §14.6.5), cast expressions (§14.6.6), unary **+**, **-**, **~**, ***** expressions, binary **+**, **-**, *****, **/**, **%**, **<<**, **>>**, **<**, **<=**, **>**, **>=**, **==**, **!=**, **is**, **as**, **&**, **|**, **^** expressions (§14.7, §14.8, §14.9, §14.10), compound assignment expressions (§14.13.2), **checked** and **unchecked** expressions (§14.5.12), array and delegate creation expressions (§14.5.10).

Each of these expressions has one or more sub-expressions that are unconditionally evaluated in a fixed order. For example, the binary **%** operator evaluates the left hand side of the operator, then the right hand side. An indexing operation evaluates the indexed expression, and then evaluates each of the index expressions, in order from left to right. For an expression *expr*, which has sub-expressions *expr₁*, *expr₂*, ..., *expr_n*, evaluated in that order:

- The definite assignment state of v at the beginning of *expr₁* is the same as the definite assignment state at the beginning of *expr*.
- The definite assignment state of v at the beginning of *expr_i* (*i* greater than one) is the same as the definite assignment state at the end of *expr_{i-1}*.
- The definite assignment state of v at the end of *expr* is the same as the definite assignment state at the end of *expr_n*.

12.3.3.21 Invocation expressions and object creation expressions

For an invocation expression *expr* of the form:

1 *primary-expression* (*arg*₁, *arg*₂, ..., *arg*_{*n*})

2 or an object creation expression of the form:

3 **new** *type* (*arg*₁, *arg*₂, ..., *arg*_{*n*})

- 4 • For an invocation expression, the definite assignment state of *v* before *primary-expression* is the
5 same as the state of *v* before *expr*.
- 6 • For an invocation expression, the definite assignment state of *v* before *arg*₁ is the same as the state of
7 *v* after *primary-expression*.
- 8 • For an object creation expression, the definite assignment state of *v* before *arg*_{*i*} is the same as the
9 state of *v* before *expr*.
- 10 • For each argument *arg*_{*i*}, the definite assignment state of *v* after *arg*_{*i*} is determined by the normal
11 expression rules, ignoring any **ref** or **out** modifiers.
- 12 • For each argument *arg*_{*i*} for any *i* greater than one, the definite assignment state of *v* before *arg*_{*i*} is the
13 same as the state of *v* after *arg*_{*i-1*}.
- 14 • If the variable *v* is passed as an **out** argument (i.e., an argument of the form “**out** *v*”) in any of the
15 arguments, then the state of *v* after *expr* is definitely assigned. Otherwise; the state of *v* after *expr* is
16 the same as the state of *v* after *arg*_{*n*}.

17 12.3.3.22 Simple assignment expressions

18 For an expression *expr* of the form *w* = *expr-rhs*:

- 19 • The definite assignment state of *v* before *expr-rhs* is the same as the definite assignment state of *v*
20 before *expr*.
- 21 • If *w* is the same variable as *v*, then the definite assignment state of *v* after *expr* is definitely assigned.
22 Otherwise, the definite assignment state of *v* after *expr* is the same as the definite assignment state of
23 *v* after *expr-rhs*.

24 12.3.3.23 && expressions

25 For an expression *expr* of the form *expr-first* && *expr-second*:

- 26 • The definite assignment state of *v* before *expr-first* is the same as the definite assignment state of *v*
27 before *expr*.
- 28 • The definite assignment state of *v* before *expr-second* is definitely assigned if the state of *v* after
29 *expr-first* is either definitely assigned or “definitely assigned after true expression”. Otherwise, it is
30 not definitely assigned.
- 31 • The definite assignment statement of *v* after *expr* is determined by:
 - 32 o If the state of *v* after *expr-first* is definitely assigned, then the state of *v* after *expr* is
33 definitely assigned.
 - 34 o Otherwise, if the state of *v* after *expr-second* is definitely assigned, and the state of *v* after
35 *expr-first* is “definitely assigned after false expression”, then the state of *v* after *expr* is
36 definitely assigned.
 - 37 o Otherwise, if the state of *v* after *expr-second* is definitely assigned or “definitely assigned
38 after true expression”, then the state of *v* after *expr* is “definitely assigned after true
39 expression”.
 - 40 o Otherwise, if the state of *v* after *expr-first* is “definitely assigned after false expression”, and
41 the state of *v* after *expr-second* is “definitely assigned after false expression”, then the state
42 of *v* after *expr* is “definitely assigned after false expression”.
 - 43 o Otherwise, the state of *v* after *expr* is not definitely assigned.

[Example: In the example

```

class A
{
    static void F(int x, int y) {
        int i;
        if (x >= 0 && (i = y) >= 0) {
            // i definitely assigned
        }
        else {
            // i not definitely assigned
        }
        // i not definitely assigned
    }
}

```

the variable *i* is considered definitely assigned in one of the embedded statements of an *if* statement but not in the other. In the *if* statement in method *F*, the variable *i* is definitely assigned in the first embedded statement because execution of the expression (*i* = *y*) always precedes execution of this embedded statement. In contrast, the variable *i* is not definitely assigned in the second embedded statement, since *x* >= 0 might have tested false, resulting in the variable *i*'s being unassigned. *end example*]

12.3.3.24 || expressions

For an expression *expr* of the form *expr-first* || *expr-second*:

- The definite assignment state of *v* before *expr-first* is the same as the definite assignment state of *v* before *expr*.
- The definite assignment state of *v* before *expr-second* is definitely assigned if the state of *v* after *expr-first* is either definitely assigned or “definitely assigned after false expression”. Otherwise, it is not definitely assigned.
- The definite assignment statement of *v* after *expr* is determined by:
 - If the state of *v* after *expr-first* is definitely assigned, then the state of *v* after *expr* is definitely assigned.
 - Otherwise, if the state of *v* after *expr-second* is definitely assigned, and the state of *v* after *expr-first* is “definitely assigned after true expression”, then the state of *v* after *expr* is definitely assigned.
 - Otherwise, if the state of *v* after *expr-second* is definitely assigned or “definitely assigned after false expression”, then the state of *v* after *expr* is “definitely assigned after false expression”.
 - Otherwise, if the state of *v* after *expr-first* is “definitely assigned after true expression”, and the state of *v* after *expr-second* is “definitely assigned after true expression”, then the state of *v* after *expr* is “definitely assigned after true expression”.
 - Otherwise, the state of *v* after *expr* is not definitely assigned.

[Example: In the example

```

class A
{
    static void G(int x, int y) {
        int i;
        if (x >= 0 || (i = y) >= 0) {
            // i not definitely assigned
        }
        else {
            // i definitely assigned
        }
        // i not definitely assigned
    }
}

```

the variable `i` is considered definitely assigned in one of the embedded statements of an `if` statement but not in the other. In the `if` statement in method `G`, the variable `i` is definitely assigned in the second embedded statement because execution of the expression `(i = y)` always precedes execution of this embedded statement. In contrast, the variable `i` is not definitely assigned in the first embedded statement, since `x >= 0` might have tested false, resulting in the variable `i`'s being unassigned. *end example*]

12.3.3.25 ! expressions

For an expression `expr` of the form `! expr-operand`:

- The definite assignment state of `v` before `expr-operand` is the same as the definite assignment state of `v` before `expr`.
- The definite assignment state of `v` after `expr` is determined by:
 - If the state of `v` after `expr-operand` is definitely assigned, then the state of `v` after `expr` is definitely assigned.
 - If the state of `v` after `expr-operand` is not definitely assigned, then the state of `v` after `expr` is not definitely assigned.
 - If the state of `v` after `expr-operand` is “definitely assigned after false expression”, then the state of `v` after `expr` is “definitely assigned after true expression”.
 - If the state of `v` after `expr-operand` is “definitely assigned after true expression”, then the state of `v` after `expr` is “definitely assigned after false expression”.

12.3.3.26 ?: expressions

For an expression `expr` of the form `expr-cond ? expr-true : expr-false`:

- The definite assignment state of `v` before `expr-cond` is the same as the state of `v` before `expr`.
- The definite assignment state of `v` before `expr-true` is definitely assigned if and only if the state of `v` after `expr-cond` is definitely assigned or “definitely assigned after true expression”.
- The definite assignment state of `v` before `expr-false` is definitely assigned if and only if the state of `v` after `expr-cond` is definitely assigned or “definitely assigned after false expression”.

12.4 Variable references

A *variable-reference* is an *expression* that is classified as a variable. A *variable-reference* denotes a storage location that can be accessed both to fetch the current value and to store a new value.

variable-reference:
expression

[*Note:* In C and C++, a *variable-reference* is known as an *lvalue*. *end note*]

12.5 Atomicity of variable references

Reads and writes of the following data types shall be atomic: `bool`, `char`, `byte`, `sbyte`, `short`, `ushort`, `uint`, `int`, `float`, and reference types. In addition, reads and writes of enum types with an underlying type in the previous list shall also be atomic. Reads and writes of other types, including `long`, `ulong`, `double`, and `decimal`, as well as user-defined types, need not be atomic. Aside from the library functions designed for that purpose, there is no guarantee of atomic read-modify-write, such as in the case of increment or decrement.

